



The Game Loop

YOU'LL SHIP A complete, playable round-based game.

7.1 Designing your game on paper

7.2 Rounds and timers

7.3 Collectibles and scoring

7.4 Upgrade shop

7.5 Sound and visual effects

Everything comes together here. You'll design Coin Rush on paper, then build the round system, coins that spawn and get collected, an upgrade shop, and the sound and visual effects that make it all feel great.

7.1 Designing your game on paper

Before writing the round script, answer these on one page:

1. **Core loop:** what does the player do over and over? *Jump between platforms collecting coins before the timer ends, without touching lava.*
2. **Session loop:** what does one round look like? *15 s intermission → 90 s round → results → repeat.*
3. **Progression:** why come back tomorrow? *Saved coins, levels, and upgrades that change how you play.*
4. **Win condition:** *Most coins collected this round wins a bonus.*
5. **First 30 seconds:** *Intro splash, menu, PLAY, you're in the lobby watching the current round with a countdown.*

Then list every system you need. For Coin Rush: round manager, coin spawner, coin collection, rewards, shop, effects. Build them in that order, testing after each.

TIP Cut features until the list scares you a little less. A small game that's finished beats a big game that isn't.

7.2 Rounds and timers

The round manager is a single **Script** in **ServerScriptService**. It publishes its state through **attributes** on **ReplicatedStorage**, which the HUD from Module 4 already listens to.

Set up the map first: a **Lobby** model with a **LobbySpawn** part, and an **Arena** folder containing two folders: **Spawns** (parts where players start a round) and **CoinSpots** (invisible, anchored parts where coins can appear).

```
-- ServerScriptService/RoundManager
local Players = game:GetService("Players")
local ReplicatedStorage = game:GetService("ReplicatedStorage")
local ServerScriptService = game:GetService("ServerScriptService")

local GameConfig = require(ReplicatedStorage.GameConfig)
local PlayerDataService = require(ServerScriptService.PlayerDataService)
local CoinSpawner = require(ServerScriptService.CoinSpawner)

local arena = workspace.Arena
```

```

local lobbySpawn = workspace.Lobby.LobbySpawn

local function setState(status: string, timeLeft: number)
    ReplicatedStorage:SetAttribute("RoundStatus", status)
    ReplicatedStorage:SetAttribute("TimeLeft", timeLeft)
end

local function teleport(player: Player, target: BasePart)
    local root = player.Character and player.Character:FindFirstChild("HumanoidRootPart")
    if root then
        root.CFrame = target.CFrame + Vector3.new(0, 4, 0)
    end
end

local function countdown(status: string, seconds: number, stopEarly: (() -> boolean)?)
    for t = seconds, 1, -1 do
        setState(status, t)
        task.wait(1)
        if stopEarly and stopEarly() then return end
    end
end

while true do
    -- Intermission: wait for at least one player
    repeat
        setState("Waiting for players", 0)
        task.wait(1)
    until #Players:GetPlayers() >= 1

    countdown("Intermission", GameConfig.IntermissionLength)

    -- Start the round
    local spawns = arena.Spawns:GetChildren()
    local roundCoins: { [Player]: number } = {}
    for i, player in Players:GetPlayers() do
        roundCoins[player] = 0
        teleport(player, spawns[(i - 1) % #spawns + 1])
    end

    CoinSpawner.start(function(player: Player)
        if roundCoins[player] then
            roundCoins[player] += 1
            PlayerDataService.addCoins(player, GameConfig.CoinValue)
        end
    end)
end

```

```

end)

countdown("Round", GameConfig.RoundLength, function()
    return #Players:GetPlayers() == 0
end)

CoinSpawner.stop()

-- Results
local winner, best = nil, -1
for player, count in roundCoins do
    if player.Parent and count > best then
        winner, best = player, count
    end
    if player.Parent then
        PlayerDataService.addXP(player, 10 + count)
    end
end
if winner then
    PlayerDataService.addCoins(winner, GameConfig.WinnerBonus)
    setState(`{winner.DisplayName} wins with {best} coins!`, 0)
else
    setState("Round over", 0)
end
task.wait(5)

for _, player in Players:GetPlayers() do
    teleport(player, lobbySpawn)
end
end
end

```

Add `WinnerBonus = 25` to `GameConfig`.

`PlayerDataService` is the ModuleScript you built from Module 6. The round manager only needs three functions from it:

```

PlayerDataService.addCoins(player, amount)    -- server-side only
PlayerDataService.addXP(player, amount)
PlayerDataService.trySpend(player, amount)    -- returns true if they could afford it

```

7.3 Collectibles and scoring

Coins are spawned **by the server**, from a template in **ServerStorage** (so clients can't copy or fake them). Build one coin: a yellow Neon cylinder rotated on its side, size about `0.4, 2.5, 2.5`, Anchored, `CanCollide = false`. Name it `Coin` and move it into `ServerStorage`.

```
-- ServerScriptService/CoinSpawner (ModuleScript)
local ServerStorage = game:GetService("ServerStorage")
local Players = game:GetService("Players")

local template = ServerStorage:WaitForChild("Coin")
local spots = workspace.Arena.CoinSpots:GetChildren()

local CoinSpawner = {}
local running = false
local active: { BasePart } = {}

local function spawnCoin(onCollect: (Player) -> ())
    local spot = spots[math.random(1, #spots)]
    local coin = template:Clone()
    coin.CFrame = spot.CFrame
    coin.Parent = workspace
    table.insert(active, coin)

    local taken = false
    coin.Touched:Connect(function(hit)
        if taken then return end
        local player = Players:GetPlayerFromCharacter(hit.Parent)
        if not player then return end
        taken = true
        onCollect(player)
        coin:Destroy()
    end)
end

function CoinSpawner.start(onCollect: (Player) -> ())
    running = true
    task.spawn(function()
        while running do
            if #active < 25 then
                spawnCoin(onCollect)
            end
            -- Forget coins that were destroyed
            for i = #active, 1, -1 do
```

```

        if not active[i].Parent then table.remove(active, i) end
    end
    task.wait(0.5)
end
end)
end

function CoinSpawner.stop()
    running = false
    for _, coin in active do coin:Destroy() end
    table.clear(active)
end

return CoinSpawner

```

Because the coin detects the touch **on the server** and uses `GetPlayerFromCharacter`, a client can't claim coins it never touched.

Spinning coins. Spinning 25 coins on the server wastes bandwidth. Spin them on each client instead, in a LocalScript in StarterPlayerScripts:

```

local RunService = game:GetService("RunService")

RunService.RenderStepped:Connect(function(dt)
    for _, coin in workspace:GetChildren() do
        if coin.Name == "Coin" and coin:IsA("BasePart") then
            coin.CFrame *= CFrame.Angles(0, dt * 3, 0)
        end
    end
end)

```

Upgrade shop

Two upgrades for Coin Rush: **Speed** (WalkSpeed 16 → 22) and **Jump** (JumpPower 50 → 60). They're bought with coins and saved in the player's `Upgrades` table.

Server (extends the secure handler from Module 5):

```

-- ServerScriptService/Shop
local Players = game:GetService("Players")
local ReplicatedStorage = game:GetService("ReplicatedStorage")
local ServerScriptService = game:GetService("ServerScriptService")

```

```

local GameConfig = require(ReplicatedStorage.GameConfig)
local PlayerDataService = require(ServerScriptService.PlayerDataService)
local BuyUpgrade = ReplicatedStorage.Remotes.BuyUpgrade

local EFFECTS = {
    Speed = function(humanoid: Humanoid) humanoid.WalkSpeed = 22 end,
    Jump = function(humanoid: Humanoid)
        humanoid.UseJumpPower = true
        humanoid.JumpPower = 60
    end,
}

local function applyUpgrades(player: Player, character: Model)
    local humanoid = character:WaitForChild("Humanoid") :: Humanoid
    local data = PlayerDataService.get(player)
    if not data then return end
    for name in data.Upgrades do
        if EFFECTS[name] then EFFECTS[name](humanoid) end
    end
end

Players.PlayerAdded:Connect(function(player)
    player.CharacterAdded:Connect(function(character)
        applyUpgrades(player, character)
    end)
end)

BuyUpgrade.OnServerEvent:Connect(function(player, name)
    if typeof(name) ~= "string" or not EFFECTS[name] then return end
    local data = PlayerDataService.get(player)
    if not data or data.Upgrades[name] then return end
    local price = GameConfig.UpgradePrices[name]
    if not PlayerDataService.trySpend(player, price) then return end

    data.Upgrades[name] = true
    if player.Character then applyUpgrades(player, player.Character) end
end)

```

Client: the SHOP button from the main menu opens a panel with one button per upgrade, each calling `BuyUpgrade:FireServer("Speed")`. Show the price from `GameConfig`, and grey out owned upgrades by reading an attribute or a RemoteFunction that returns the player's upgrades.

Sound and visual effects

Feedback turns "it works" into "it feels amazing". Add these:

Coin pickup (client-side, so it's instant). The server tells the collecting player with a RemoteEvent `CoinCollected`, passing the coin's position:

```
-- In CoinSpawner, right before coin:Destroy():
ReplicatedStorage.Remotes.CoinCollected:FireClient(player, coin.Position)
```

```
-- StarterPlayerScripts/CoinEffects (LocalScript)
local ReplicatedStorage = game:GetService("ReplicatedStorage")
local SoundService = game:GetService("SoundService")
local Debris = game:GetService("Debris")

local pickupSound = SoundService:WaitForChild("CoinPickup")

ReplicatedStorage.Remotes.CoinCollected.OnClientEvent:Connect(function(position: Vector3)
    pickupSound.PlaybackSpeed = 0.95 + math.random() * 0.15 -- small variation feels
    natural
    SoundService:PlayLocalSound(pickupSound)

    local burst = Instance.new("Part")
    burst.Anchored, burst.CanCollide, burst.Transparency = true, false, 1
    burst.Position = position
    burst.Parent = workspace

    local sparks = Instance.new("ParticleEmitter")
    sparks.Color = ColorSequence.new(Color3.fromRGB(255, 220, 60))
    sparks.LightEmission = 1
    sparks.Speed = NumberRange.new(8, 14)
    sparks.Lifetime = NumberRange.new(0.3, 0.6)
    sparks.SpreadAngle = Vector2.new(180, 180)
    sparks.Size = NumberSequence.new(0.4, 0)
    sparks.Parent = burst
    sparks:Emit(20)

    Debris:AddItem(burst, 1)
end)
```

More juice to add:

- A countdown beep for the last 5 seconds of a round, and a horn when it ends.

- Lava bubbling: a `ParticleEmitter` on the lava floor plus a low looping sound.
- Camera shake or a red screen flash when a player falls in lava.
- Background music in lobby vs. round (two `Sound` objects in `SoundService`, crossfaded with `TweenService` on `Volume`).

Find sounds in the Toolbox **Audio** tab or the Creator Store. Use Roblox-provided or properly licensed audio only.

TRY IT

1. Play a full round with a friend (or with **Test** → **Clients and Servers** → **2 players**).
2. Tune `RoundLength`, coin count and upgrade prices in `GameConfig` until rounds feel tense but fair.

Module recap

- ☐ One-page design with core loop, session loop and progression
- ☐ `RoundManager` cycles intermission → round → results
- ☐ Server-spawned coins, collected securely, with a winner bonus
- ☐ Speed and Jump upgrades bought with coins and saved
- ☐ Pickup sound and particles, plus at least two other effects